Lab 5 — Submission Document

GIX Equipment Tracker + Events App

TECHIN510 — University of Washington / GIX

Student	Proud Dumrongpun
Course	TECHIN510
Equipment App	https://510-lab5-peerayad.streamlit.app/
Events App	https://lab-5-peerayad-event.streamlit.app/
GitHub (Classroom)	https://github.com/GIX-Luyao/lab-5-peerayad
Tech Stack	Streamlit + Supabase (PostgreSQL) + Python

Component A — Staff Interview

A.1 Interview Notes

Field	Detail
Interviewee	Maason Kao
Role	Equipment Checkout & Returns Manager, GIX
Topic	Equipment check-in and return workflow, pain points

Context

GIX tracks all equipment (laptops, cameras, microphones, cables) using BlueTally — a digital inventory system. Each physical item gets an 8-digit asset tag sticker as its unique ID. Students can log in to see what is available.

Key findings

- **Slow manual check-in:** Staff must go through returned items one by one, look up each item in BlueTally manually, and type in the Asset ID and Product Number. This is very slow.
- **Long Amazon product names:** Amazon product descriptions are too long to use directly as item names in BlueTally. Staff must manually shorten every name before entry.
- **Unattended returns:** When no staff is present, students leave equipment on the desk with no one to log it. This happens frequently with cameras.
- **External barcode generation:** Barcodes cannot be generated inside BlueTally. A separate pre-generation step is required before items can be added.

- **Underused CSV import:** There is a CSV file with all items and BlueTally has an import function, but it has only been used once.

Key quotes

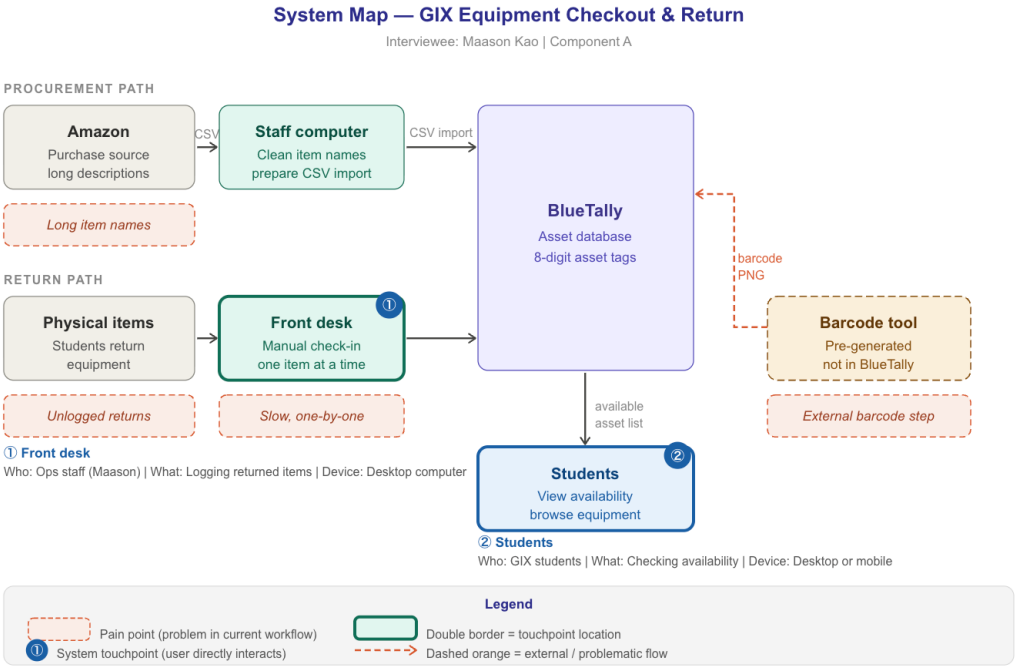
> *Is there a better way to automate the check-in system? A team comes to us, goes through the list one by one... that process is very slow.*
>
> *Barcodes are not generated on BlueTally — there is a pre-generation step. You have to tap in the information manually.*
>
> *The friction is the item name — the Amazon descriptions are so long. Don't want this to be the item name.*
>
> *Sometimes when nobody is in, they just put it on the desk — no one is here. Happens a lot with cameras.*

A.2 Build Mandate

> *Based on the interview, I will build an equipment check-in and borrow request system because the interviewee said the current process of going through items one by one and manually typing everything into BlueTally is very slow, which means the system needs to reduce manual data entry, auto-generate barcodes for asset tags, support unattended returns through student self-reporting, and give staff a condition report before they physically inspect returned items.*

A.3 System Map

The diagram below shows the two main data flows — procurement and returns — both converging into BlueTally, with pain points (coral boxes) and touchpoints annotated.



A.4 System Touchpoints

#	Who	What they are doing	Device
① Front Desk	Operations staff (Maason/Kevin)	Logging returned items — checking each item against BlueTally and purchase records one by one	Desktop computer at front desk
② Students	GIX students	Checking what equipment is currently available to borrow via BlueTally	Desktop or mobile device

Component B — Lab

B.1 Tech Stack Justification

> *I chose Streamlit + Supabase because this is an internal GIX staff tool where the primary pain point is data entry speed, not public scalability — and I implemented role-based authentication and multi-user persistence directly in Streamlit to meet the workflow requirements without the overhead of Next.js.*

B.2 Supabase Schema

Table: equipment

Column	Type	Notes
id	bigserial	Primary key
name	text NOT NULL	Short item name — not full Amazon description
category	text NOT NULL	Camera, Audio, Laptop, Cable, Accessory, Other
status	text	Only 'available' or 'checked_out'
notes	text	Additional info
asset_tag	text UNIQUE	Sequential 8-digit number e.g. 00000001
returned_at	timestamptz	Timestamp of last return

Table: borrow_requests

Column	Type	Notes
id	bigserial	Primary key
equipment_id	bigint	Foreign key → equipment.id
asset_tag	text	8-digit tag at request time
equipment_name	text	Name snapshot at request time
student_name	text NOT NULL	Full name of requester
requested_at	timestamptz	When submitted
approver_name	text	Staff who approved
approved_at	timestamptz	When approved
returned_at	timestamptz	When student marked returned
confirmer_name	text	Staff who confirmed return
confirmed_at	timestamptz	When confirmed
status	text	pending_approval → approved → pending_return → returned

Table: users

Column	Type	Notes
id	bigserial	Primary key
username	text UNIQUE	Login username
password_hash	text	bcrypt hashed password
full_name	text	Display name

Column	Type	Notes
role	text	'requester' or 'approver'

Table: equipment_checklist

Column	Type	Notes
id	bigserial	Primary key
equipment_id	bigint	Foreign key → equipment.id
item_name	text	Name of included accessory
description	text	Optional description

Table: return_report

Column	Type	Notes
id	bigserial	Primary key
request_id	bigint	Foreign key → borrow_requests.id
checklist_item_id	bigint	Foreign key → equipment_checklist.id
item_name	text	Name of item being reported
condition	text	'good', 'damaged', or 'missing'
note	text	Optional note from student
reported_at	timestamptz	When submitted

B.3 Responsive Design Summary

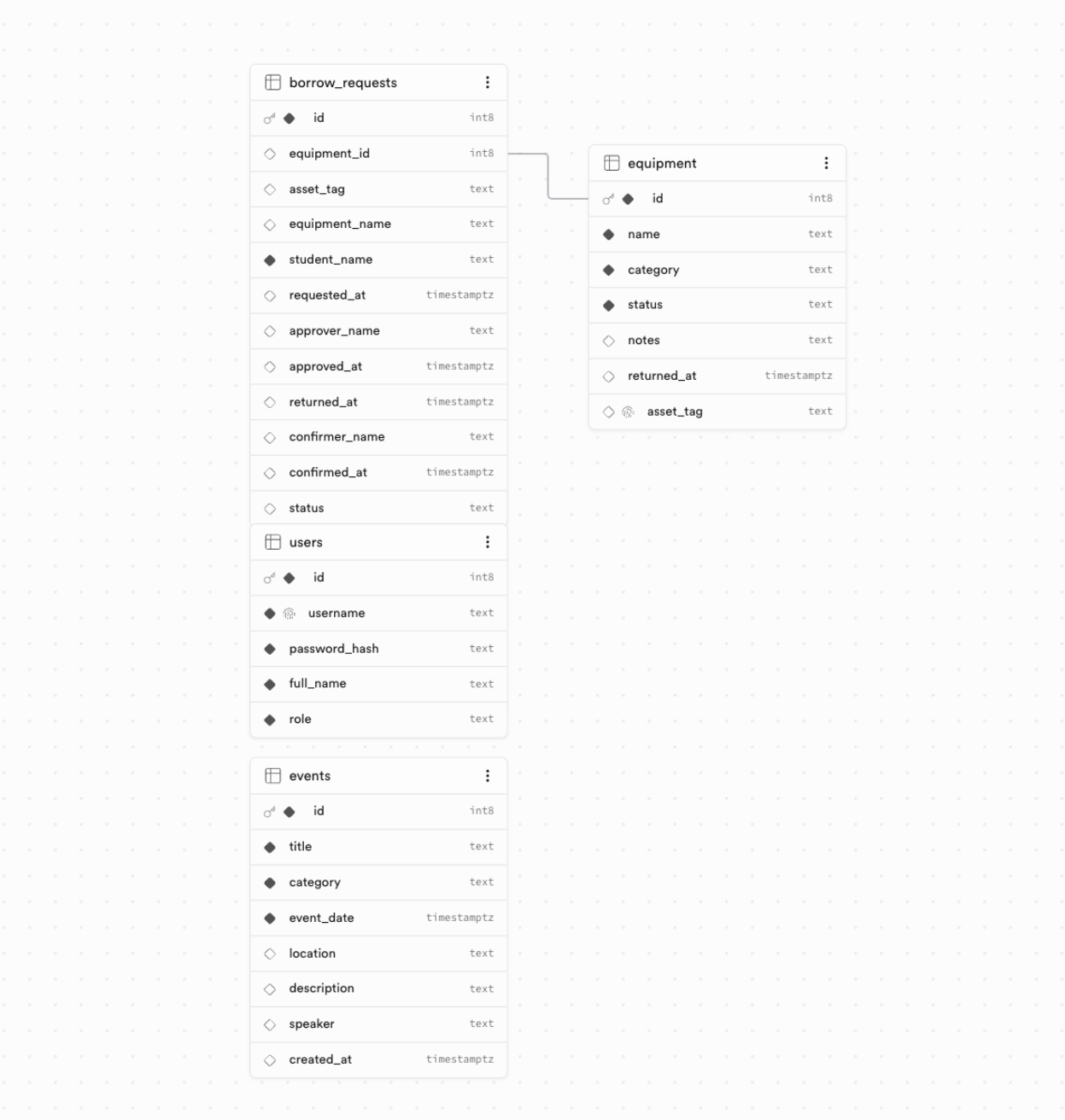
Element	Works at Phone Width?	Notes / Fix
Page title	✓ Yes	Fully visible and readable
Navigation tabs	✓ Fixed	Was cut off — fixed with CSS flex-wrap: wrap
Forms	✓ Yes	Dropdowns and inputs stack vertically on narrow screens
Tables	✓ Yes	Horizontally scrollable on narrow screens
Buttons	✓ Yes	Large enough to tap with a finger

Element	Works at Phone Width?	Notes / Fix
Text	✓ Yes	Readable without zooming

Issue found: Navigation tabs were cut off on narrow phone screens — only 3 of 6 tabs were visible.
Fix applied: Added CSS flex-wrap: wrap to the Streamlit tab list so all tabs wrap to a second row on mobile.

B.4 Deployment

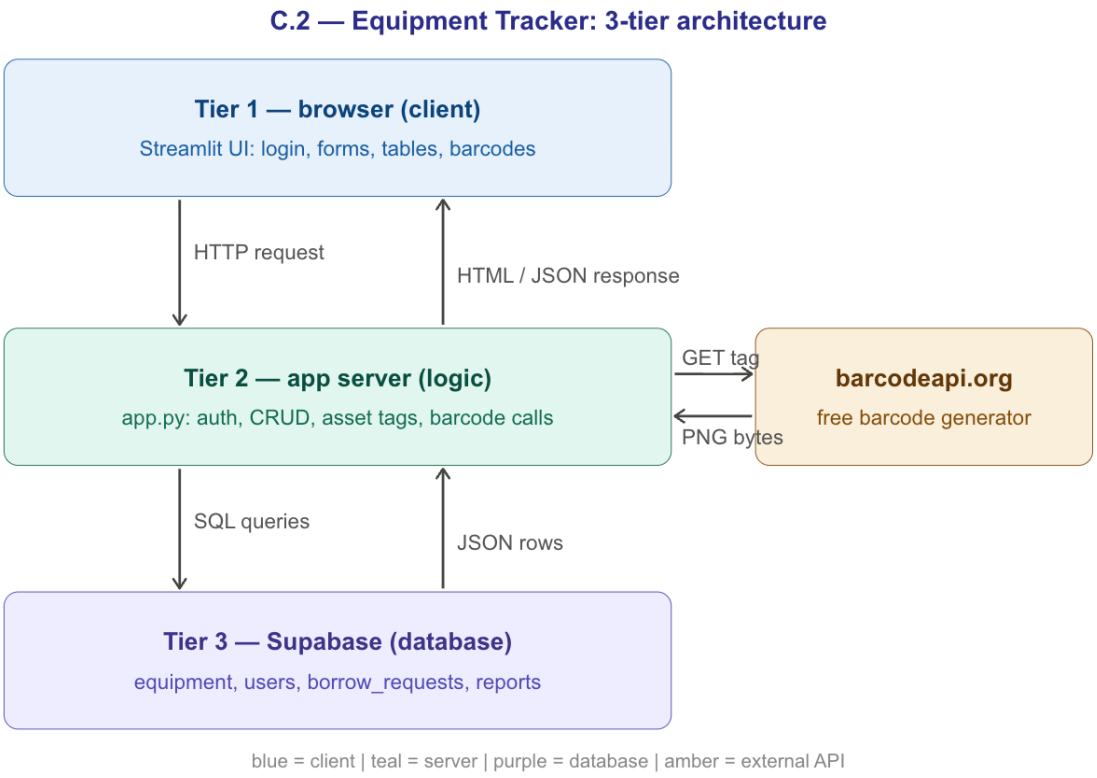
Field	Detail
Platform	Streamlit Community Cloud
Equipment Tracker URL	https://510-lab5-peerayad.streamlit.app/
Events App URL	https://lab-5-peerayad-event.streamlit.app/
GitHub (Classroom)	https://github.com/GIX-Luyao/lab-5-peerayad
Secrets	Stored in Streamlit Cloud Advanced Settings → Secrets (TOML)
Verified	No API keys or Supabase keys exposed in any source file


borrow_requests
id int8
equipment_id int8
asset_tag text
equipment_name text
student_name text
requested_at timestamptz
approver_name text
approved_at timestamptz
returned_at timestamptz
confirmer_name text
confirmed_at timestamptz
status text
users
id int8
username text
password_hash text
full_name text
role text
events
id int8
title text
category text
event_date timestamptz
location text
description text
speaker text
created_at timestamptz
equipment
id int8
name text
category text
status text
notes text
returned_at timestamptz
asset_tag text

Component C — System Architecture & Design

C.2 Three-Tier Architecture Diagram (Component B App)

This diagram is for the Equipment Tracker app (app.py) which integrates Supabase + the barcodeapi.org external API.



External API (barcodeapi.org) → called from Tier 2 → returns PNG barcode image

C.3 Design Decision Log

Field	Entry
Decision	Use sequential 8-digit asset tags (00000001, 00000002...) instead of random numbers
Alternatives considered	Random 8-digit numbers; UUIDs; letting staff manually enter tags each time
Why I chose this	Maason said tags must be exactly 8 digits. Sequential numbers are predictable — staff can see how many items exist and what was added recently just from the tag number
Trade-off	Sequential tags reveal inventory size. A random tag would be harder to guess. For a small internal GIX tool this is acceptable

Field	Entry
When would you choose differently?	If the system were public-facing or required security through obscurity, random tags would be more appropriate

Component D — Testing & Validation

D.1 Validation Results — Barcodeapi.org (External API)

#	Test Case	Input	Expected	Actual	Status Code	Pass/Fail
1	Valid input	GET barcodeapi.org/api/128/00000001 with valid 8-digit tag	200 OK, PNG image returned	200 OK — received PNG image bytes, Content-Type: image/png	200	✓ Pass
2	Invalid input	GET barcodeapi.org/api/128/INVALID!@# with special characters	400 or error response	API returned 400 — AssertionError caught: 'Barcode API returned status 400'	400	✓ Pass
3	Third invalid variant (timeout)	requests.get() with timeout=0.001 (near zero timeout)	Timeout exception caught gracefully	requests.exceptions.Timeout caught — returned {'error': 'Barcode API timed out after 5 seconds'}	N/A	✓ Pass

Note: barcodeapi.org does not require authentication, so a missing-auth test is not applicable. A timeout test was used as the third scenario instead.

D.2 External API Assert Statements (barcodeapi.org)

Located in generate_barcode() function in app.py:

```
# Assert 1 — API must return HTTP 200
assert response.status_code == 200, f'Barcode API returned status {response.status_code}'
```

```
# Assert 2 — Response must contain image data (not empty)
assert len(response.content) > 0, 'Barcode API returned empty content'
```

```
# Assert 3 — Content-Type must be an image
assert response.headers.get('Content-Type', '').startswith('image/'), \
    f"Expected image content, got {response.headers.get('Content-Type')}"
```

D.3 Error Handling Notes

D.1 Scenario	Handled Gracefully?	Notes
Scenario 1 — Valid input	Yes	Barcode image displayed correctly using st.image(io.BytesIO(barcode))
Scenario 2 — Invalid input (bad tag)	Yes	AssertionError caught, st.warning() shown to user — app continues running
Scenario 3 — Timeout (third invalid)	Yes	requests.exceptions.Timeout caught separately, user-friendly error message shown

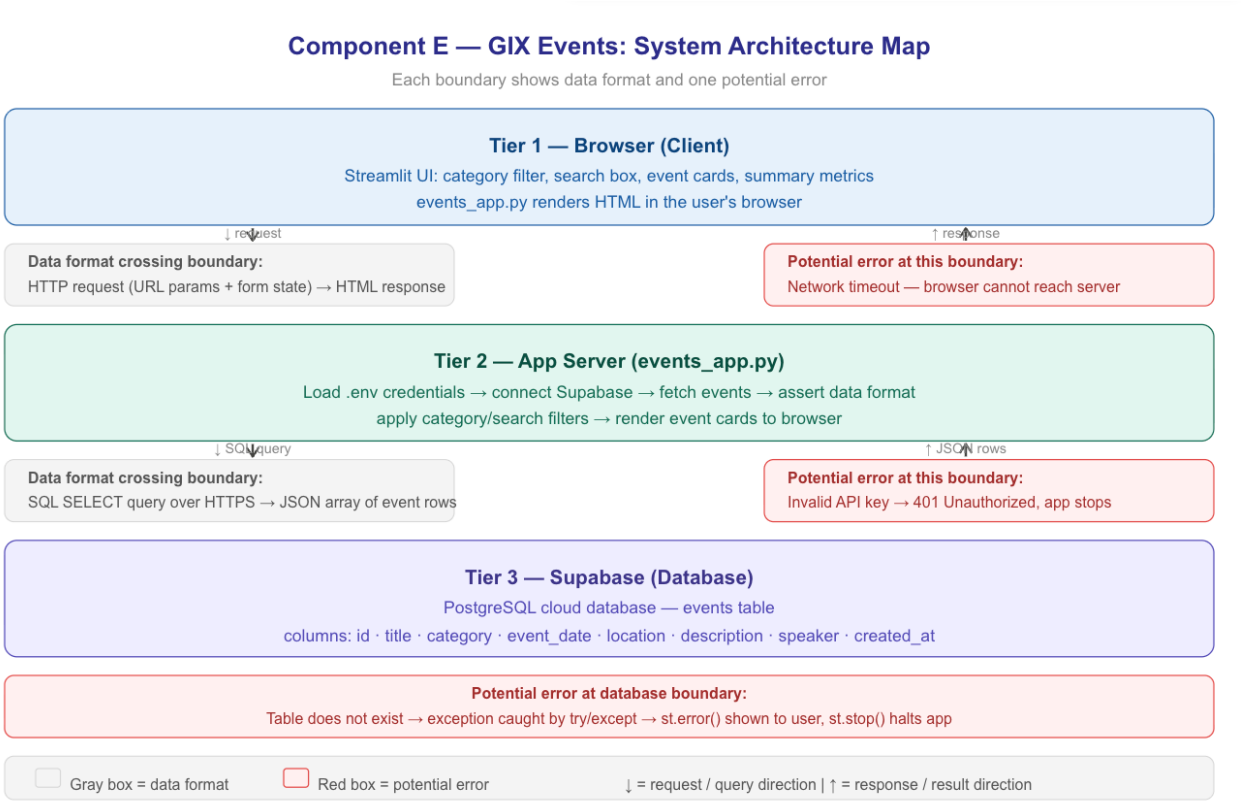
Component E — The API Connector: GIX Events

E.1 Working Events UI

Field	Detail
Live URL	https://lab-5-peerayad-event.streamlit.app/
File	events_app.py in repo root
Database	Supabase events table (same project as Equipment Tracker)
Features	Browse events • Category filter • Search • Summary metrics • Event cards
Sample data	8 events across 4 categories: Guest Lecture, Career Panel, Workshop, Social

E.2 System Architecture Map

Each boundary shows the data format crossing it and one potential error.



E.3 Testing Write-Up

Assert statements (events_app.py)

```
# Assert 1 — response must be a list
assert isinstance(events, list), 'Expected a list of events from Supabase'
```

```
# Assert 2 — each event must have required fields
for event in events:
    assert 'title' in event, 'Event missing required field: title'
    assert 'category' in event, 'Event missing required field: category'
    assert 'event_date' in event, 'Event missing required field: event_date'
```

Error scenario tests

#	Test Case	What I did	Expected	Actual	Pass/Fail
1	Valid input	Opened live app, selected Workshop from category filter	Only Workshop events shown (2 items)	App showed 2 Workshop events: Python for Data Science and Hardware Prototyping 101. Metrics updated correctly.	✓ Pass
2	Invalid input — no results	Typed 'xyzabc' in the search box	'No events match your filters.' shown, Showing: 0	App showed 'No events match your filters.' with Showing: 0 and Next Event: —. No crash.	✓ Pass
3	Wrong API key	Changed SUPABASE_KEY to 'wrongkey123' in Streamlit Cloud secrets	App shows error and stops	App showed '❌ Could not connect to database: Invalid API key' and stopped rendering.	✓ Pass

E.4 Security Verification

- No API keys or Supabase credentials hardcoded in events_app.py or any source file
- SUPABASE_URL and SUPABASE_KEY loaded via os.getenv() after load_dotenv()
- .env file is listed in .gitignore — never committed to GitHub
- Secrets stored in Streamlit Cloud Advanced Settings → Secrets (TOML format)
- Verified: searching GitHub source files shows no exposed credentials

AI Usage Log

Three AI interactions from this lab session:

Interaction 1 — Building the initial Streamlit app

Field	Detail
What I prompted	Build a Streamlit app to track GIX equipment with Supabase — browse inventory, check items in and out, add new items
What it produced	Full app.py with three tabs: Browse Equipment, Check In/Out, Add New Item — connected to Supabase with error handling
AI assumption	The AI assumed this was a single-user tool with no authentication needed
Failure mode	The app had no login — any person could edit or delete equipment without any access control
What I would change	Specify upfront: 'multi-user system with two roles: requester (student) and approver (staff)' before generating the first version

Interaction 2 — Adding login and role-based access

Field	Detail
What I prompted	Add a login system with requester and approver roles — requesters see only their own requests
What it produced	bcrypt login, session state management, role-based tab rendering, separate views per role
AI assumption	Assumed only two roles were needed and that users are pre-created manually via SQL
Failure mode	No admin role for managing users — all users must be added directly via SQL. No self-registration flow for new students.
What I would change	Specify all roles and permissions explicitly before generating, including how new users are created

Interaction 3 — Replacing weather API with barcode generation

Field	Detail
What I prompted	Replace the weather API with a barcode generation API that connects to Maason's actual pain point about external barcode generation
What it produced	generate_barcode() function using barcodeapi.org, show_barcode() helper, barcode display after adding items, reprint menu in Browse tab
AI assumption	Assumed the barcode API would always return an image and that the asset tag format (8 digits) would always be valid input

Field	Detail
Failure mode	If barcodeapi.org is down or rate-limits the app, the barcode silently fails. The assert checks status code but not rate limiting (429).
What I would change	Add explicit handling for 429 Too Many Requests and a fallback message directing staff to manually print from barcodeapi.org directly

Reflection

1.The biggest surprise was how much structure gets added even when staying with Streamlit. Just connecting Supabase introduced a genuine 3-tier architecture that simply did not exist in earlier single-file apps. The moment data needs to persist across users and sessions, the system becomes meaningfully distributed, and decisions like where to call st.stop() suddenly have real consequences for how tabs render.

2.The system map turned out to be more useful than the interview questions alone. Drawing the boxes and arrows made visible something the interview could not easily surface: the real bottleneck is the gap between when items physically arrive at the desk and when they get logged. Maason could describe the process as slow, but the map made it clear why — there is no digital step between 'student drops item on desk' and 'staff opens BlueTally.' The JTBD statement captures the user's goal, but the system map shows exactly where the friction lives in the workflow.

3.Streamlit is the right tool when you need a functional internal tool quickly for a small known team where the workflow is data-heavy and Python-centric. A good example is this GIX equipment tracker — the users are staff and students in one building, the data model is straightforward, and speed of development matters. Next.js + Supabase is the right tool when you need a public-facing product with complex interactive UI, fine-grained client-side state, or many concurrent users. A GIX-wide public equipment portal that any student discovers without staff onboarding would need Next.js — the authentication, routing, and responsive design requirements alone would outgrow what Streamlit handles gracefully.